Technical Details of The Algorithm Little OaldresPuzzle_Cryptic

Twilight-Dream

June 7, 2023

1 Abstract

This paper introduces a symmetric sequence cryptographic algorithm, "Little OaldresPuzzle_Cryptic". The algorithm combines the speed of symmetric cryptography with an extra layer of security through sequence-based encryption. Central to its design is a cryptographically secure pseudo-random number generator, "XorConstantRotation" (XCR), which greatly increases the complexity of the encryption process. The paper explores the algorithm's technical aspects, illustrating its capability to balance speed and security in high-speed data transmission scenarios.

Contents

1 Abstract **1**

2 Introduction **1**

3 Text content **2**

4 Flow of the algorithm **2**

5 Data Tests and Special Characteristics **4**
5.1 Environment Setup 5
5.2 Running the Tests 5
5.3 Unique Features and Precautions 5
5.4 Interpreting the Results 5
5.5 Contributions 5

6 Additional Quantum-Resistant OaldresPuzzle_Cryptic Block Cipher Algorithm **5**

7 Conclusion **6**

2 Introduction

This paper presents "Little OaldresPuzzle_Cryptic", a symmetric sequence cryptographic algorithm tailored for the demands of our digital age. As data generation and consumption rates increase, so do threats to data integrity and security. This scenario requires cryptographic algorithms capable of ensuring secure data exchange and high-speed performance.

Our algorithm uses the same cryptographic key for both encryption and decryption, offering significant speed advantages over asymmetric alternatives. The sequence cryptographic approach adds complexity to the encryption and decryption process, making it harder for unauthorized entities to interpret encrypted data.

The algorithm combines speed and security, leveraging a cryptographically secure pseudo-random number generator (CSPRNG), "XorConstantRotation" (XCR), which generates highly unpredictable number sequences.

The remainder of this paper will examine the algorithm's mechanics, demonstrating how it employs mathematical constants, bitwise operations, and sequence-based cryptography to ensure speed and security in data transmission.

3 Text content

This paper delves into the intricate mechanics of the "Little OaldresPuzzle_Cryptic" encryption algorithm. The algorithm rests firmly on the foundation of complex mathematical principles and operations, such as bitwise operations, hexadecimal representation, irrational and transcendental numbers, sequence generation, and pseudo-random number generation.

First, We need to specifically outline an algorithm for generator — computing the hexadecimal representation of the XCR round constants, which serves as a cornerstone for our cryptographic functions. This algorithm leverages the inherent unpredictability of several mathematical constants, including Euler's number, Pi, the Golden ratio, the square roots of 2 and 3, the cube roots of 2 and 3, Euler-Mascheroni constant, Feigenbaum constant, and the Plastic number. These constants, known for their irrational and transcendental characteristics, inject a fundamental element of randomness, thus augmenting the cryptographic strength of our algorithm. This binary sequence of constants, generated by a nonlinear Boolean function, is further segmented into 64-bit groups and returned as a hexadecimal string.

Second, we elucidate our Cryptographically Secure Pseudo-Random Number Generator (CSPRNG), termed as "XorConstantRotation" or "XCR". The XCR algorithm heavily depends on this constant data produced by the aforementioned process.

Subsequently, The heart of the discussion is devoted to our symmetric sequence cryptographic algorithm, the "Little OaldresPuzzle_Cryptic". The algorithm, designed as a class, employs the XCR algorithm for its operations. In the coming sections, the paper lays out pseudo-codes for these algorithms, bringing their mechanics to light.

4 Flow of the algorithm

Algorithm 1 Generator - Computing XCR Round Constant Hexadecimal Representation

function GenerateRoundConstant ▷ This function generates the round constant
  $e \leftarrow 2.718281828459045235360287471352662497757247093699959574966967627724076630353547594571382178525166427$ ▷ Euler's number
  $\pi \leftarrow 3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 8$ ▷ Pi
  $\phi \leftarrow 1.618033988749894848204586834365638117720309179805762862135448622705260462818902449707207204189391137$ ▷ Golden ratio $\frac{1+\sqrt{5}}{2}$
  $\sqrt{2} \leftarrow 1.414213562373095048801688724209698078569671875376948073176679737990732478462107038850387534327641573$ ▷ Square root of 2
  $\sqrt{3} \leftarrow 1.732050807568877293527446341505872366942805253810380628055806979451933016908800037081146186757248576$ ▷ Square root of 3
  $\gamma \leftarrow 0.5772156649$ ▷ Euler-Mascheroni constant
  $\delta \leftarrow 4.6692016091$ ▷ Feigenbaum constant
  $\rho \leftarrow 1.3247179572$ ▷ Plastic number $\sqrt[3]{\frac{9+\sqrt{69}}{18}} + \sqrt[3]{\frac{9-\sqrt{69}}{18}}$

  $x \leftarrow 1$
  $binary_string \leftarrow$ ""
  for $index \leftarrow 0$ **to** 140 **do**
    $result \leftarrow (e^x - \cos(\pi x)) \times (\phi x^2 - \phi x - 1) \times (\sqrt[2]{x} - \lfloor \sqrt[2]{x} \rfloor) \times (\sqrt[3]{x} - \lfloor \sqrt[3]{x} \rfloor) \times \ln(1+x) \times (x\delta - \lfloor x\delta \rfloor) \times (x\rho - \lfloor x\rho \rfloor)$
    $fractional_part \leftarrow result - \lfloor result \rfloor$ ▷ Isolate the fractional part
    $binary_fractional_part \leftarrow$ Binary of $(fractional_part \times 2^{128})$
    $hexadecimal_fractional_part \leftarrow$ Hexadecimal of $(fractional_part \times 2^{128})$
    $integer_part \leftarrow \lfloor result \rfloor$

```
        x ← x + 1
        binary_string ← binary_string.APPEND(binary_fractional_part)
    end for
    integer_value ← Integer of binary_string
    hexadecimal_string ← Hexadecimal of integer_value
    return hexadecimal_string          ▷ Return the hexadecimal string of the round constant
end function
```

Algorithm 2 Cryptographically Secure Pseudo-Random Number Generator - XorConstantRotation (XCR)

```
function XCR INITIALIZE(seed)
    seed ∈ F_2^64
    state ∈ F_2^64
    x, y ∈ F_2^64
    ROUND_CONSTANT_i ∈ F_2^64
    x, y = 0
    state = seed                                   ▷ Initial state is the seed
end function

function XCR GENERATION(round)
    y ← (x ⊕ (state ⋘ 32)) ⊕ (state ⋘ 19))
    if x == 0 then
        x ← ROUND_CONSTANT[round (mod size(ROUND_CONSTANT)])
    else
        x ← x ⊞_64 ((x ⋘ 7) ⊕ ROUND_CONSTANT[round (mod size(ROUND_CONSTANT)]))
    end if
    state ← (x ⊕ y) ⊞_64 state
    return y
end function
```

Algorithm 3 LittleOaldresPuzzle_Cryptic Class Implementation

```
prng                                           ▷ XorConstantRotation (XCR) Instance
function ENCRYPTIONCOREFUNCTION(data, key, round)
    subkey ← key ⊕ PRNG(round)                 ▷ Generate subkey using PRNG
    choice_function ← PRNG(subkey ⊕ key) (mod 4)          ▷ Generate choice function
    bit_rotation_amount ← PRNG(subkey ⊕ choice_function) (mod 64)   ▷ Calculate bit rotation
amount
    if choice_function = 0 then
        result ← data ⊕ subkey                 ▷ XOR operation
    else if choice_function = 1 then
        result ← ¬(data ⊕ subkey)              ▷ Negation(Bitwise NOT) after XOR operation
    else if choice_function = 2 then
        result ← (data ⋘ bit_rotation_amount)  ▷ Left bitwise rotation
    else if choice_function = 3 then
        result ← (data ⋙ bit_rotation_amount)  ▷ Right bitwise rotation
    end if
    result ← result ⊕ (1 ≪ bit_rotation_amount)
    result ← result ⊞_64 ((key ⋙ 3) ⊕ (subkey ⋙ 11))
    return result
end function
function DECRYPTIONCOREFUNCTION(data, key, round)
    subkey ← key ⊕ PRNG(round)                 ▷ Generate subkey using PRNG
    choice_function ← PRNG(subkey ⊕ key) (mod 4)          ▷ Generate choice function
    bit_rotation_amount ← PRNG(subkey ⊕ choice_function) (mod 64)   ▷ Calculate bit rotation
amount
    result ← data ⊟_64 ((key ⋙ 3) ⊕ (subkey ⋙ 11))
```

```
    result ← result ⊕ (1 ≪ bit_rotation_amount)
    if choice_function = 0 then
        result ← result ⊕ subkey                          ▷ XOR operation
    else if choice_function = 1 then
        result ← ¬result                                  ▷ Negation(Bitwise NOT)
        result ← result ⊕ subkey                          ▷ XOR operation
    else if choice_function = 2 then
        result ← (data ⋘ bit_rotation_amount)             ▷ Left bitwise rotation
    else if choice_function = 3 then
        result ← (data ⋙ bit_rotation_amount)             ▷ Right bitwise rotation
    end if
    return result
end function
function ResetPRNG
    prng.Seed(seed)                                       ▷ Reset the PRNG with a new seed
end function
```

We believe the readers are acquainted with the three closely related algorithms aforementioned. However, they might now be grappling with several questions regarding their composition and purpose. Thus, it is essential to address these queries and provide a more profound understanding of our design intentions.

The three algorithms, specifically, The **XCR Round Constant Generator**, Cryptographically Secure Pseudo-Random Number Generator - **XorConstantRotation (XCR)**, and **LittleOaldresPuzzle_Cryptic**, are not standalone entities but integral components of our cryptographic architecture. Each algorithm serves a purpose, and they function in unison to create a secure, robust cryptographic system.

The **XCR** Round Constant Generator lays the foundation for our cryptographic functions. This generator algorithm employs a fascinating assortment of mathematical constants, each known for its irrational and transcendental nature. This rich mix instills a degree of randomness and complexity that strengthens the cryptographic robustness of our algorithm.

Next is the Cryptographically Secure Pseudo-Random Number Generator - **XorConstantRotation (XCR)**. This algorithm is the backbone of our cryptographic system, generating sequences that exhibit randomness on the surface yet are entirely predictable given the known seed. We designed the **XCR** algorithm to leverage two-bit operations: XOR and rotating-left. These operations exhibit exceptional confusion and diffusion properties, effectively scattering changes in the input evenly across the output. This feature significantly enhances the system's security.

The final piece of the puzzle is the **LittleOaldresPuzzle_Cryptic** algorithm. This algorithm is a symmetric sequential cipher algorithm that is dependent on **XCR**. It encrypts input data through a series of complex bit operations, each adding a layer of complexity and security. The key to the steps is the generation of a subkey using **XCR**, which drives the choice of operation to be performed on the data. The decryption function of **LittleOaldresPuzzle_Cryptic** mirrors the steps of the encryption function, effectively undoing all operations executed during the encryption phase.

In this design, we purposefully harnessed certain inherent characteristics of mathematics and computer science, such as non-linearity, confusion, diffusion, and pseudorandom number generation. The aim was to create a cryptographic system that offers enhanced security, unpredictability, and resistance to attacks. Together, these components provide a formidable foundation for cryptographic protection. As such, our system stands as a testament to the strength of intertwining various elements of mathematics and computer science in creating a robust, secure cryptographic system.

5 Data Tests and Special Characteristics

The repository encompasses not only the OaldresPuzzle_Cryptic algorithm discussed in this paper but also its quantum-resistant counterpart, designed for future cryptographic needs. Despite being slower in packet encryption and decryption (based on test data of 10MB and a key size of 5120 bytes), it contributes to the overall resilience of the cryptographic system. This section is intended to guide users in testing these algorithms using the code available in the repository.

Link: README.md

5.1 Environment Setup

Before you begin, ensure you have an environment compatible with the code. Clone the repository using the following link:

XorConstantRotation.cpp
LittleOaldresPuzzle_Cryptic.cpp
gist

Next, install the required software and libraries as stated in the README.md file..

5.2 Running the Tests

Within the repository, you will find two primary directories, *OOP* and *Template*. The *OOP* directory offers an object-oriented version of the algorithm, ideal for developers familiar with object-oriented programming. Conversely, the *Template* directory presents a simplified implementation, suitable for beginners or those seeking a more straightforward understanding of the algorithm.

Choose the implementation that suits your requirements, navigate to the appropriate directory, and follow the README.md instructions to compile and execute the tests. These tests will furnish a holistic understanding of the algorithm's cryptographic robustness, speed, and overall performance.

5.3 Unique Features and Precautions

It's crucial to note the peculiar characteristics and necessary precautions while using these algorithms. For instance, encryption and decryption operations demand a reset of the internal key state after each use. Therefore, if an encryption operation follows a decryption operation (or vice versa), the internal key state must be reset first to ensure correct functioning.

5.4 Interpreting the Results

Once the tests conclude, you will obtain a set of numerical performance metrics. These results, available in the README.md file, enable a performance comparison between the Algorithm Little OaldresPuzzle_Cryptic and other prevalent algorithms. However, while comparing, remember that an algorithm's efficacy isn't only about speed but also about security. Hence, consider these test results thoughtfully, factoring in the complexity of the sequences generated by the algorithm and its resilience against attacks.

5.5 Contributions

We value your feedback and contributions. If you encounter possible improvements or any issues, feel free to submit a pull request or open an issue in the GitHub repository.

6 Additional Quantum-Resistant OaldresPuzzle_Cryptic Block Cipher Algorithm

This repository houses an additional robust block cipher algorithm, developed independently from the symmetric sequence algorithm that is the primary focus of this paper. Conceived with the anticipation of future cryptographic challenges, particularly those presented by quantum computing, this block cipher algorithm offers several defining characteristics. It is designed to mitigate risks associated with brute force attacks, withstand analytical attacks on the key, and resist potential quantum computer intrusions.

Although slower in encrypting and decrypting packets (as evidenced by tests with 10MB data packets and a 5120-byte key, which required approximately one and a half minutes to execute), the algorithm confers a significant advantage by future-proofing cryptographic systems against potential advancements in quantum computing. This symmetric block cipher cryptographic algorithm has been tailored to meet contemporary cryptographic requirements. It prioritizes unpredictability and a high level of analytical complexity, making it suitable for managing protected, large-scale binary data files while ensuring requisite cryptographic robustness.

For a more comprehensive understanding, detailed information regarding the implementation and unique characteristics of this block cipher algorithm can be found in the corresponding directory of the repository. Despite being outside the primary scope of this paper, which is dedicated to the symmetric sequence algorithm, we encourage interested readers and researchers to explore this quantum-resistant block cipher algorithm and its potential applications.

7 Conclusion

This paper presents a novel symmetric sequence cryptographic algorithm that demonstrates high-speed encryption and decryption capabilities and robust security features. In our era of rapidly evolving digital needs, this algorithm stands out as an innovative tool capable of managing large-scale binary data files securely and efficiently. Additionally, the quantum-resistant block cipher algorithm included in the same repository further underscores our commitment to staying ahead of the curve in the face of future cryptographic challenges.

While this paper primarily delves into the symmetric sequence algorithm, the repository presents a holistic view of our cryptographic contributions. We encourage interested readers and researchers to delve into both algorithms and contribute their insights and potential enhancements. In the dynamic field of cryptography, collaborative efforts and continuous innovation are vital to staying prepared for future challenges.